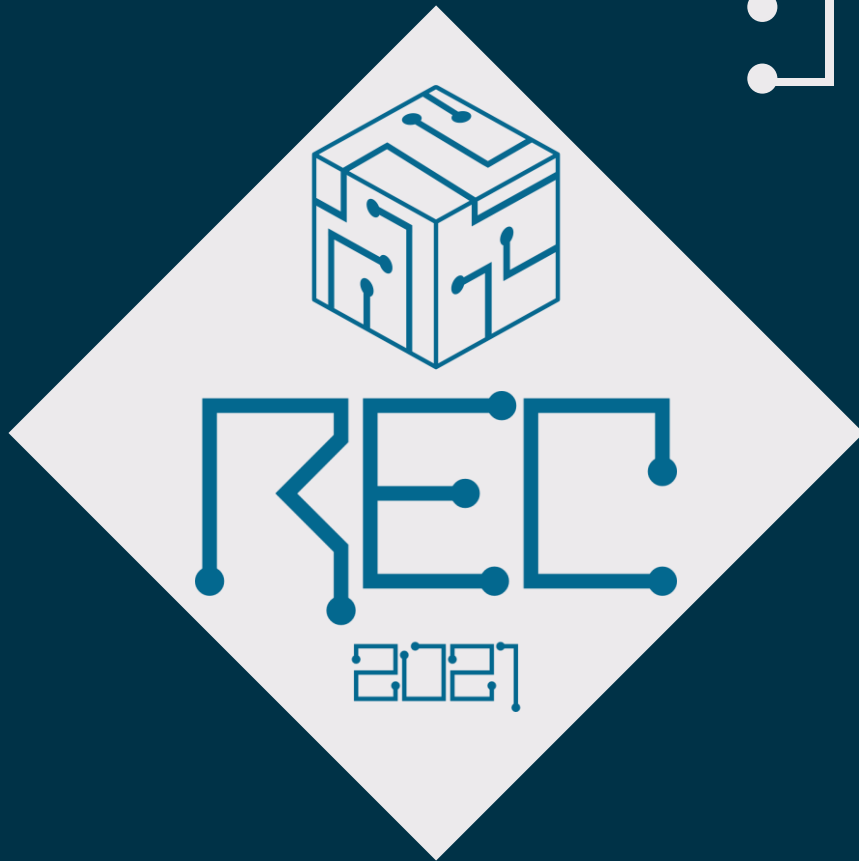


Automatic Selection and Insertion of HLS Directives via a Source-to-source Compiler

Tiago Santos
João M. P. Cardoso

July 6, 2021

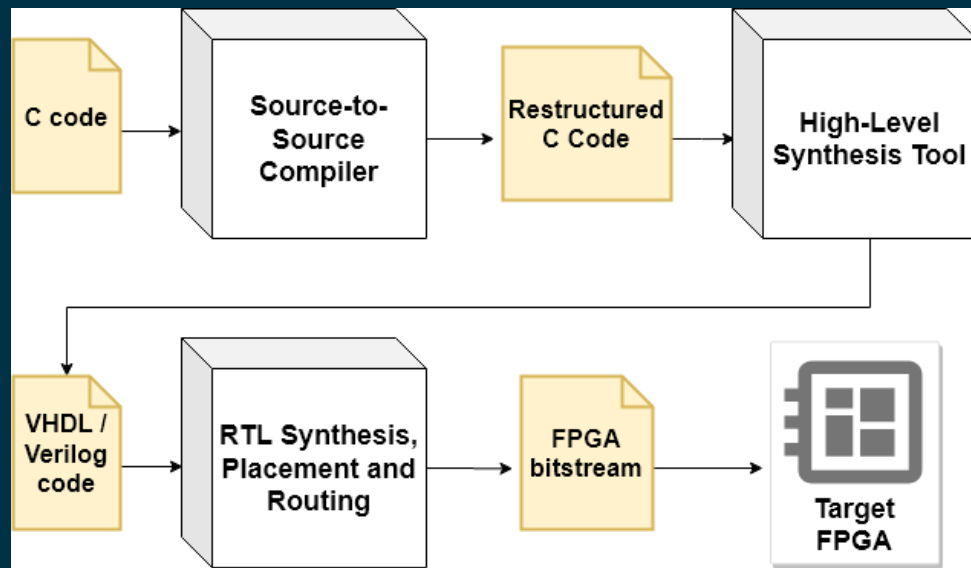




Outline

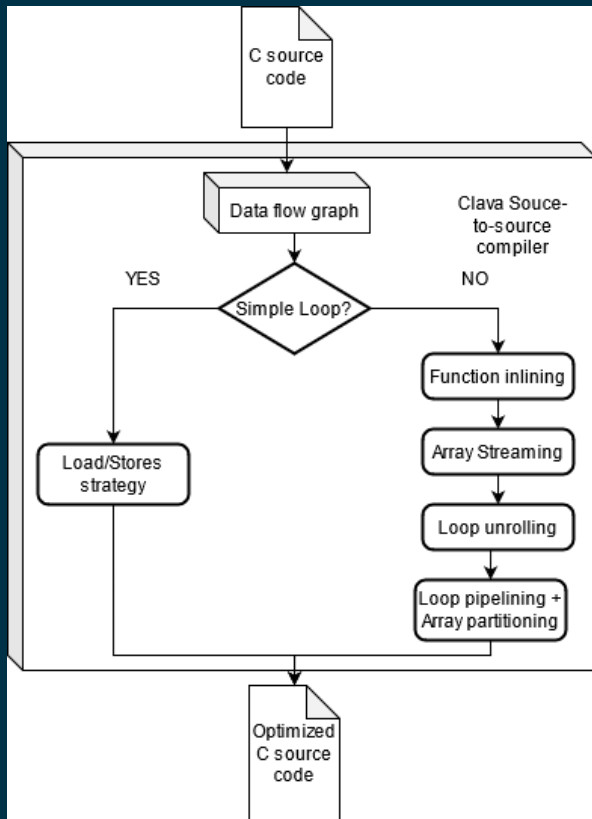
- Context and motivation
- Our approach
- Code Analysis Framework
- Optimization Strategies
- Experimental Results
- Related Work
- Conclusion

Motivation



- High-Level Synthesis is of paramount importance for software developers to program FPGAs
- However, code needs to be optimized first, which requires expert knowledge
- Automatic code restructuring and the use of HLS directives are required for HLS to be viable for common software developers - this can be done using source-to-source compilers!

Our Approach



- Selection and configuration of Vivado HLS directives based on simple and controllable heuristics (e.g., akin to those of *gcc*)
- Heuristics guided by features extracted from DFGs
- Generic approach that fits multiple domains
- Integrated on the Clang-based Clava C/C++ to C/C++ compiler



Our Approach

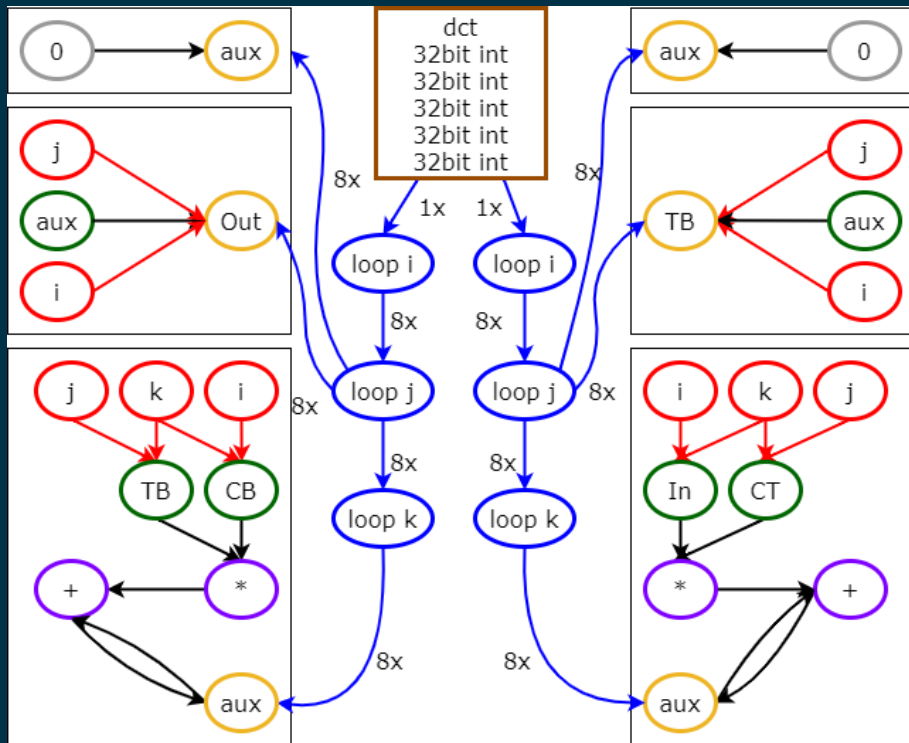
```
void computeGradient(float grad[N], float feature[N], int scale)
{
    for (int i = 0; i < N; i++)
        grad[i] = scale * feature[i];
}
```



```
void computeGradient(float grad[N], float feature[N], int scale)
{
    #pragma HLS array_partition variable=grad cyclic factor=32
    #pragma HLS array_partition variable=feature cyclic factor=32
    for (int i = 0; i < N; i++)
        #pragma HLS unroll factor=32
        #pragma HLS pipeline
        grad[i] = scale * feature[i];
}
```

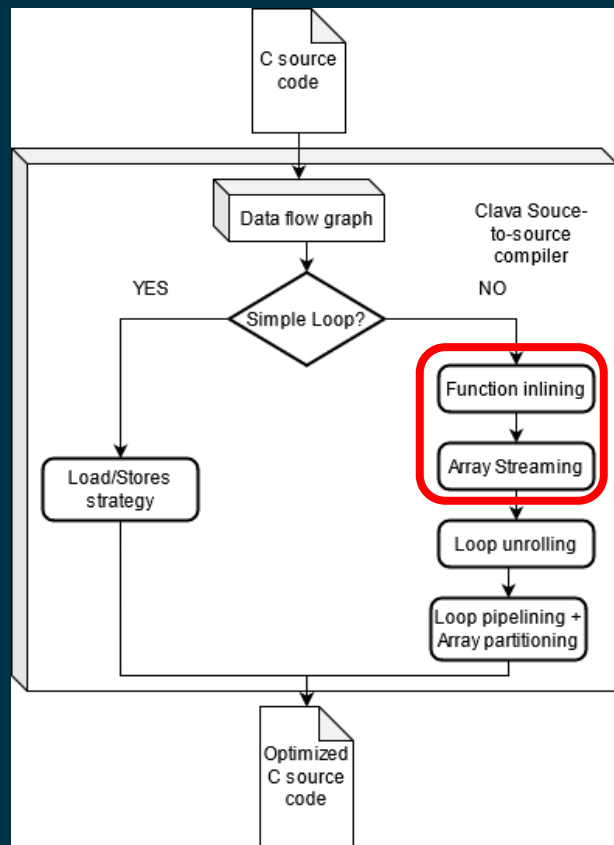
- Selection and configuration of Vivado HLS directives based on simple and controllable heuristics (e.g., akin to those of gcc)
- Heuristics guided by features extracted from DFGs
- Generic approach that fits multiple domains
- Integrated on the Clang-based Clava C/C++ to C/C++ compiler

Code Analysis Framework



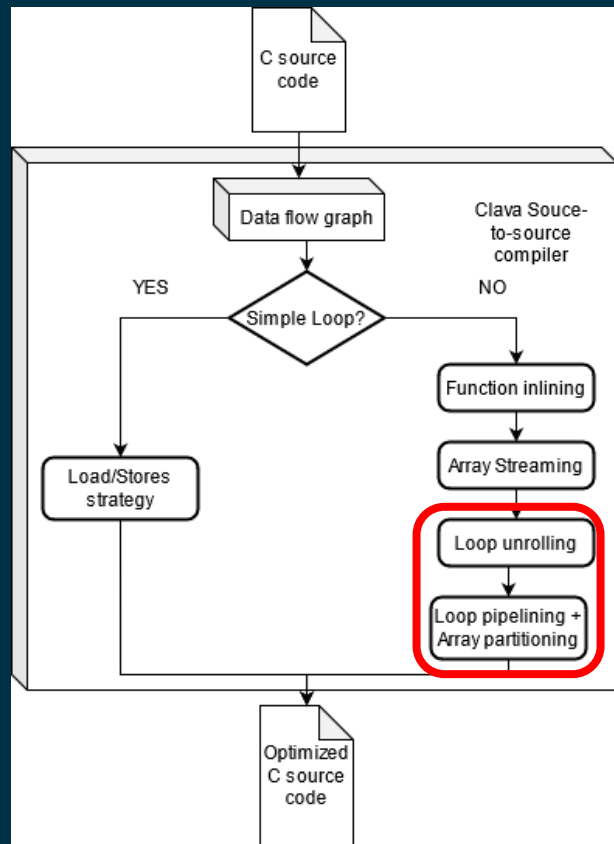
- Data Flow Graph (DFG) generated from the Clava AST and CFG
- Hierarchy reflects repeated patterns caused by loops and function calls
- Subgraphs are calculated by using a store of a variable/array as the root
- Analysis made by extracting multiple features, such as the number of load/stores, iterations and operations in each subgraph

Optimization Strategies (1)



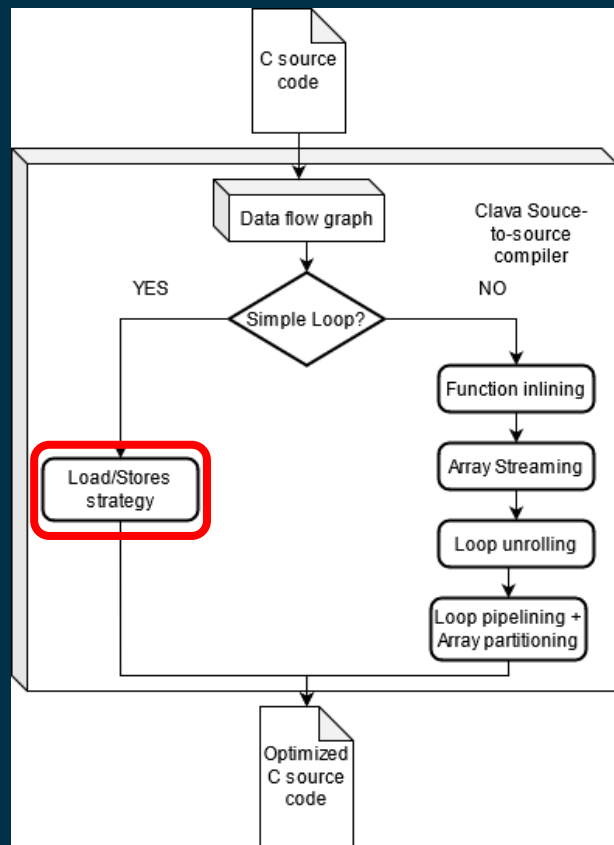
- **Function Inlining:** balances the cost of the called function vs. the cost of the callee
 - $mainCost > \frac{callFrequency * functionCost}{B}$
 - Cost is defined by the number of array load operations in the function
 - B makes the heuristic more permissive/restrictive, and is configurable by the user (default is B=2)
- **Array Streaming:** Arrays accessed sequentially can be implemented as FIFOs

Optimization Strategies (2)



- Loop unrolling and Loop pipelining: applied to nested loops, based on the number of iterations of all the loops in the nest
- Resource constraint directives are applied to limit the resource usage of DSPs
- Array partitioning:
 - Smaller arrays are mapped to registers
 - Larger arrays are partitioned

Optimization Strategies (3)



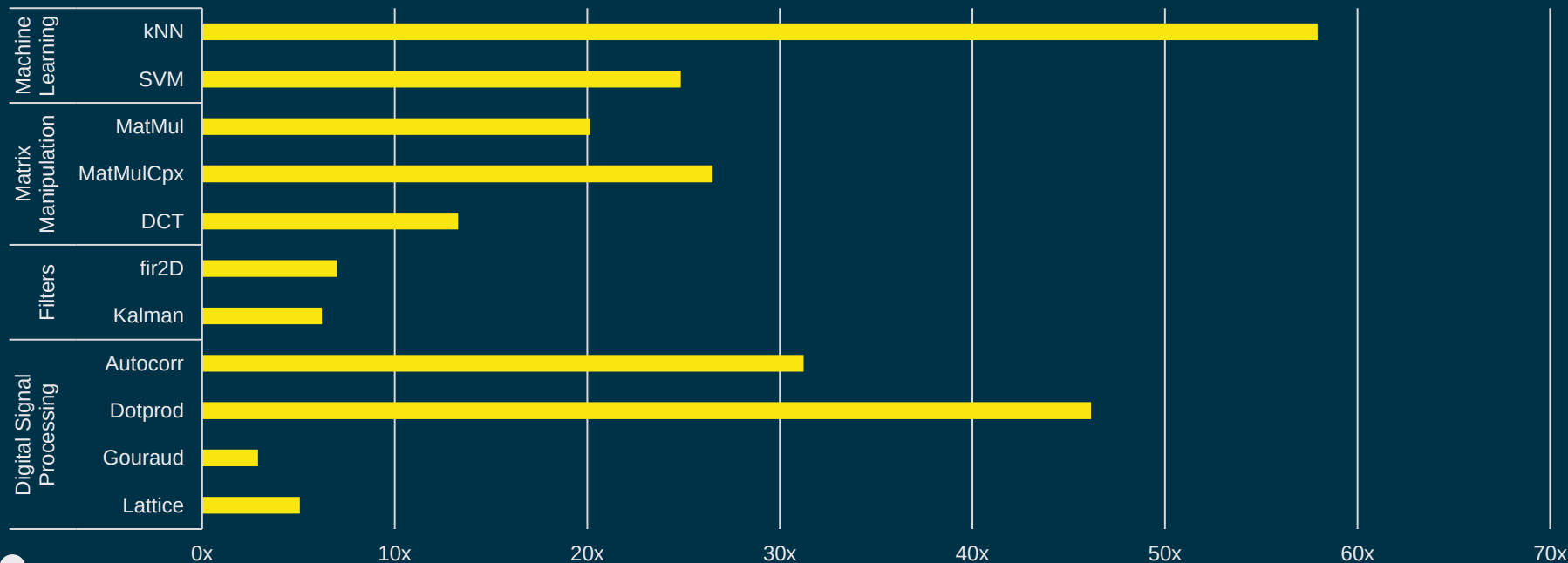
- Load/stores strategy: exclusive to simple loops (no nesting, only one read/write per array, per iteration)
- Cyclic array partitioning and unrolling by the same factor (user-configurable)
- Loop pipelining is also applied

Experimental Setup

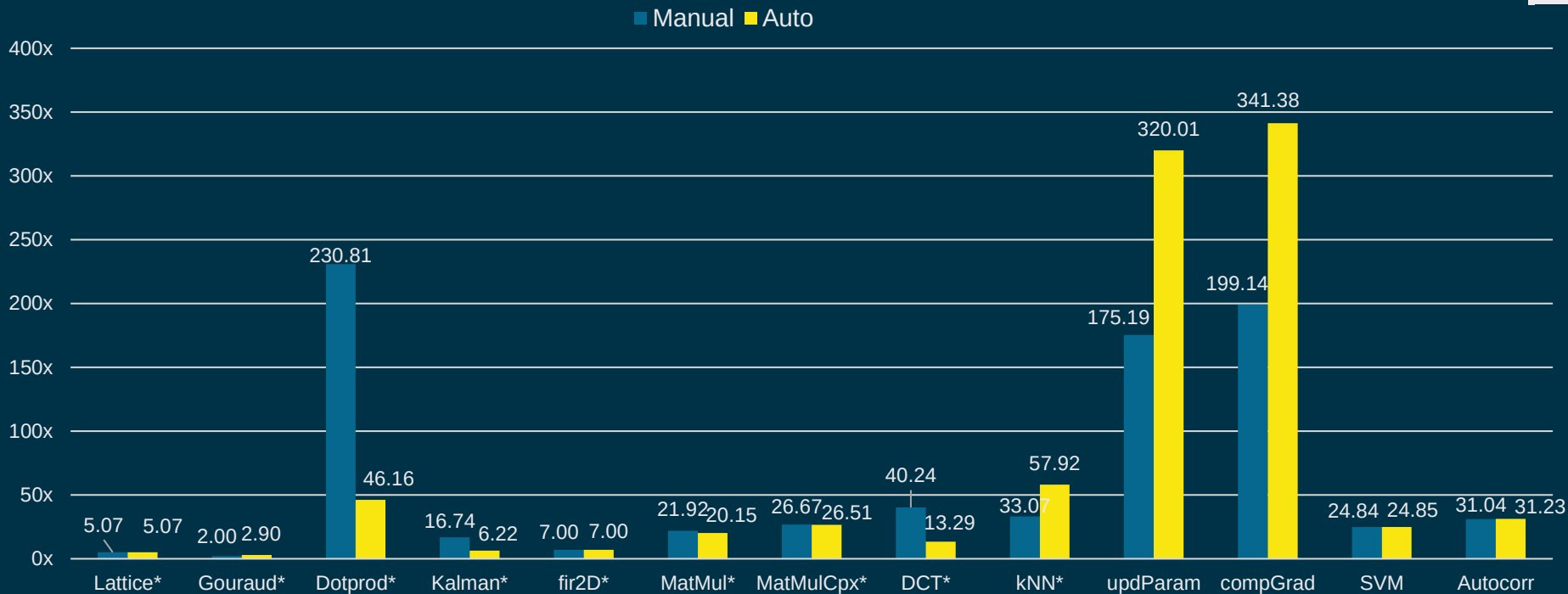
- HLS applied using Vivado HLS 2019.1 and targeting a Xilinx Artix-7 FPGA
- Four sets of benchmarks from multiple sources:
 - Machine Learning (k-Nearest Neighbours, Support Vector Machine)
 - Matrix Manipulation (Real and complex multiplication, Discrete Cosine Transform)
 - Filter-based Kernels (FIR, Kalman)
 - Digital Signal Processing kernels (autocorrelation, dot product, Gouraud shading, lattice filter)

Experimental Results

Latency speedup achieved in comparison to the unoptimized versions



Comparison to Manual Optimization

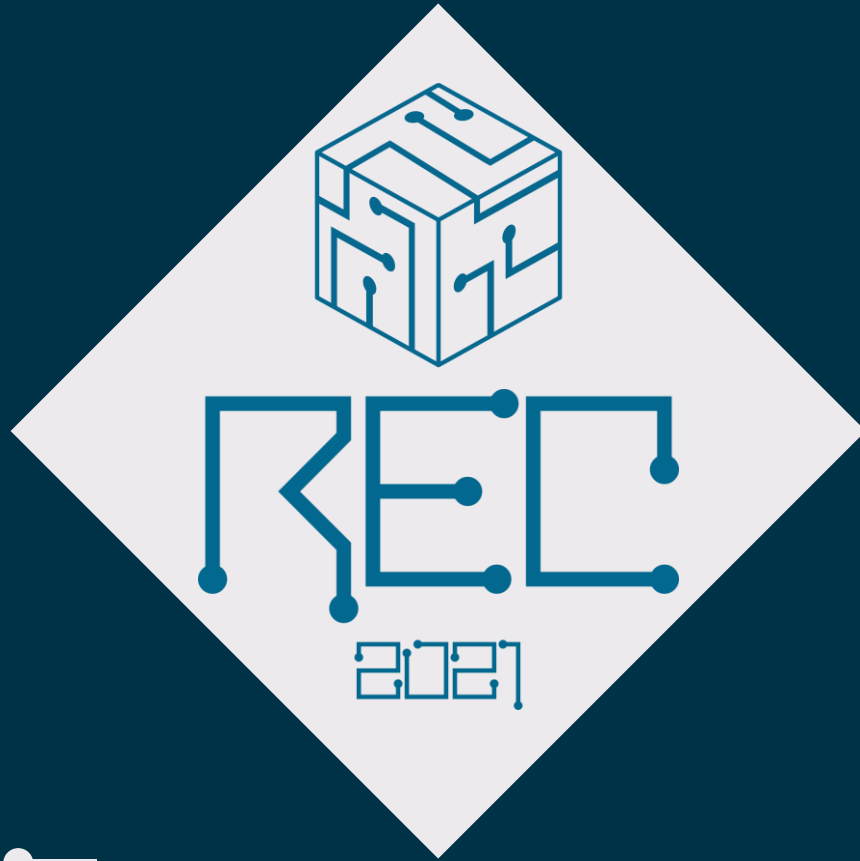


Related Work

- Design-space exploration [Tsoutsouras et al., JSPS, 2017]
- Trace-based graph restructuring [Ferreira and Cardoso, FSP'2018 and ARC' 2019]
- Ordered loop transformations [Li et al., JRTIP, 2017]
- Aspect-oriented programming to specify compilation strategies [Cardoso et al., IEEE FCCM, 2012]

Conclusion

- New source-to-source framework to optimize code for FPGAs using simple yet effective strategies
- Optimization done through the selection, configuration and insertion of Vivado HLS directives
- Benchmarks achieved interesting results when compared to manually optimized versions
- Results published in the 2020 International Conference on Field-Programmable Technology (ICFPT)



Thank you!

Any questions?